

GCP Free-Tier Setup Guide — Dev-Utills (SQL Hub)

Deploy the FastAPI backend + React frontend + Postgres on one always-free GCP `e2-micro` VM with Docker Compose + Caddy. Do every step below in your web browser unless marked `SSH`.

What you need before starting: a Google account, a credit/debit card for identity verification (no charge within free-tier limits), and your `Dev-Utills` repo cloned or ready to clone.

Free-tier guardrails (stay at \$0/mo):

Resource	Free limit	Rule
VM	1× <code>e2-micro</code> , full month (~744 h)	Region must be <code>us-west1</code> , <code>us-central1</code> , or <code>us-east1</code>
Disk	30 GB standard persistent disk	Do not add extra disks, snapshots, or SSD
Egress	1 GB/mo out of North America	Keep <code>yt-dlp</code> media downloads light
Extras in plan	Cloud Storage 5 GB regional, Artifact Registry 0.5 GB, Cloud Build 2,500 min, Secret Manager 6 versions	Enough for backups + images

Part A — Account, project, billing (browser)

Step 1 — Sign up for Google Cloud

1. Go to <https://console.cloud.google.com/freetrial>.
2. Sign in with your Google account.
3. Enter your payment method for verification (temporary \$0–\$1 hold, no charge).
4. Accept the trial — you get **\$300 credit for 90 days** plus the always-free tier.

Step 2 — Create a project

1. Open <https://console.cloud.google.com/projectcreate>.

2. Project name: `sql-hub-prod` (ID can be auto-generated, e.g. `sql-hub-prod-123456`).
3. Click **Create** and wait for the notification that it is ready.
4. Select it in the project picker (top bar).

Step 3 — Confirm the billing account

1. Go to **Billing** → <https://console.cloud.google.com/billing>.
2. Confirm your billing account is linked to `sql-hub-prod`.
3. Status reads `Active` / `Free trial`. Nothing is billed while inside free limits.

Step 4 — Set a \$1 budget alert (stops surprises)

1. Go to **Billing** → **Budgets & alerts** → **Create budget**.
2. Scope: project `sql-hub-prod`, amount: `$1.00`.
3. Alert thresholds: 50%, 90%, 100% → email yourself.
4. Save. This only notifies; it never shuts anything down.

Step 5 — Enable the Compute Engine API

1. Go to <https://console.cloud.google.com/apis/library/compute.googleapis.com>.
2. Confirm project `sql-hub-prod` is selected.
3. Click **Enable** and wait for the checkmark.

Part B — Create the free VM (browser)

Step 6 — Create the `e2-micro` instance

1. Go to **Compute Engine** → **VM instances** → <https://console.cloud.google.com/compute/instances>.
2. Click **Create instance**.
3. Set exactly:
4. **Name:** `sql-hub-vm`
5. **Region:** `us-central1` (Iowa) — always-free region
6. **Zone:** `us-central1-a`
7. **Machine type:** `E2` → `e2-micro` (2 vCPU shared, 1 GB RAM)
8. **Boot disk:** Debian 12 (Bookworm), **Standard persistent disk, 30 GB**
9. **Firewall:** check **Allow HTTP traffic** and **Allow HTTPS traffic**
10. Click **Create** and wait for the green checkmark.

11. Note the **External IP** column — that is your server address.

Step 7 — Reserve a static IP (optional, keeps IP after reboot)

1. Go to **VPC network** → **IP addresses** → <https://console.cloud.google.com/networking/addresses>.
2. Click **Reserve external static IP**, attach it to `sql-hub-vm`.
3. Static IPs attached to a running free VM are free; unattached ones are billed — always keep it attached.

Part C — Prepare the VM (SSH-in-browser)

Step 8 — Open SSH in the browser

1. Back on the **VM instances** page, click the **SSH** button next to `sql-hub-vm`.
2. A browser popup terminal opens — you are now logged into the VM. All commands below run here.

Step 9 — Install Docker + Compose plugin

```
sudo apt-get update && sudo apt-get install -y \
  ca-certificates curl gnupg
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/debian/gpg \
  | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
echo "deb [signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/debian bookworm stable" \
  | sudo tee /etc/apt/sources.list.d/docker.list
sudo apt-get update && sudo apt-get install -y \
  docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
sudo usermod -aG docker $USER
# Close the SSH window and click SSH again so the group applies:
docker --version && docker compose version
```

Step 10 — Clone the repo and configure the backend env

```
git clone <YOUR-REPO-URL> ~/Dev-Utils && cd ~/Dev-Utils
cp backend/.env.example backend/.env
```

```
nano backend/.env
```

Set these keys in `backend/.env` :

```
DATABASE_URL=postgresql+psycpg2://sqlhub:CHANGE_ME@db:5432/sqlhub
AUTH_ENABLED=true
ALLOW_SIGNUP=true
CORS_ORIGINS=http://<VM-EXTERNAL-IP>,https://<YOUR-DOMAIN-IF-ANY>
JWT_SECRET=<paste-a-long-random-string-here>
JWT_EXPIRE_DAYS=7
```

Generate a secret with:

```
python3 -c "import secrets; print(secrets.token_urlsafe(48))"
```

Single-user mode: set `AUTH_ENABLED=false` only if the app will never be public.

Part D — Deploy and verify (browser + SSH)

Step 11 — Build and start the stack (SSH)

```
cd ~/Dev-Utils
# Requires docker-compose.prod.yml with db + api + web(Caddy) services:
docker compose -f docker-compose.prod.yml up -d --build
docker compose -f docker-compose.prod.yml ps
curl -s http://127.0.0.1/health
```

Expected: `{"ok": true, ...}` . If not, check `docker compose logs api` .

Step 12 — Verify in the browser

1. Open `http://<VM-EXTERNAL-IP>/` — the React UI loads (served from `frontend/dist` by Caddy).
2. Open `http://<VM-EXTERNAL-IP>/health` — backend health JSON.
3. Log in, create a section/query, restart the stack (`docker compose restart`), and confirm the data is still there (Postgres volume works).

4. With a domain: point DNS `A` record at the external IP, set Caddy's site address to the domain for auto-TLS, and re-check `https://<YOUR-DOMAIN>/` .

Step 13 — Stay free (browser, monthly 2-minute check)

1. Open **Billing** → **Reports**, filter to `sql-hub-prod` — confirm `$0.00` .
2. Confirm the VM is still `e2-micro` in `us-central1` , disk ≤ 30 GB standard.
3. Watch egress (1 GB/mo cap) — heavy `yt-dlp` media downloads are the main risk.

Cleanup (if you ever want to stop)

1. SSH: `docker compose -f docker-compose.prod.yml down -v` (the `-v` deletes DB data — export first).
2. Console: **Compute Engine** → **VM instances** → delete `sql-hub-vm` .
3. **VPC** → **IP addresses** → release the static IP.
4. Optional: delete the project to remove every trace.

Generated 2026-09-13 for the Dev-Utils always-on GCP plan. Repo root: `GCP_SETUP_GUIDE.md` + `GCP_SETUP_GUIDE.pdf` .